COMS 3009A
Software Design

Requirements



University of the Witwatersrand, Johannesburg

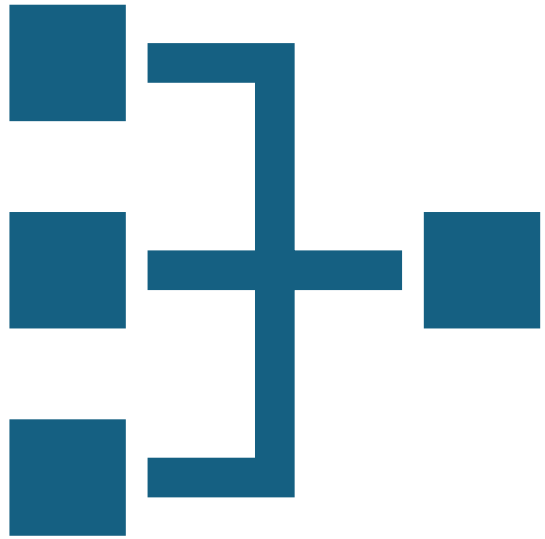
School of Computer Science and Applied Mathematics

Lucky Nkosi

What is a software requirement?

What is a Software Requirement?

- A **requirement** specifies the business **functions** that the user will be able to perform using the system-to-be in different “situations” or “contexts”, and the kind of **experience** the user will have during this work
  - May include concerns, such as how the system will manage the resources (computing, network,…), how the system will manage and protect user’s data, etc.



Client : Why don't you clearly understand my requirements ?

His Requirement :



Software Requirements

User requirements will often be high-level, vague and incomplete. They are more like high-level goals, or business goals, rather than software requirements needed by the developer



How do we bridge the gap?

Software engineer’s job to translate requirements into something useful

Vague to technical requirements

Ask: What matters? What are the important considerations?

- Features
- Security
- Data Sovereignty
- Availability and Reliability?
- User Personas

Understanding what matters

helps us understand “scenarios” or “situations” we need to cater for and relevant parameters (system inputs).

Understanding scenarios and their parameters help us decide what the system should do or respond to the situation.

- Eg: ATM user needs to be Authenticated with parameters being a card and pin.
- Situation is Authentication and parameters are the card and pin

This understanding is unpacked into tasks.

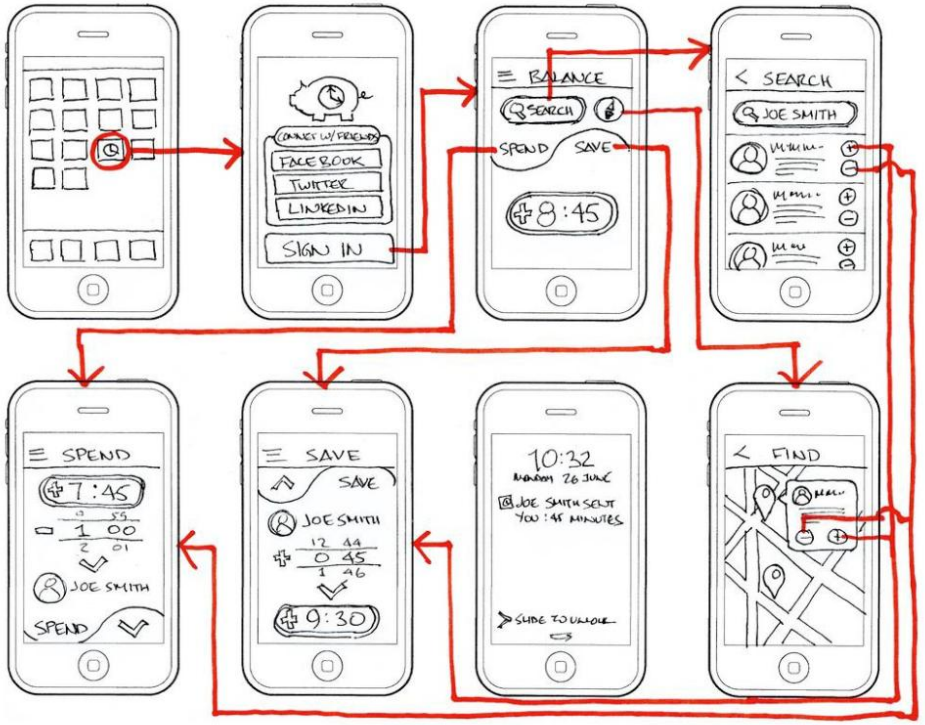
Understanding
what matters is called
Requirements Engineering

Types of Requirements

- **Functional requirements:** Product features/ features essential to user's task completion
- **Non-functional requirements** (AKA Quality requirements)
  - Usability (ease of use, efficiency, learning curve, etc)
  - Security (encryption, protection from viruses/malware, privacy, etc.)
  - Reliability (bug-free, not crashing, no hardware failures, etc.)
  - Performance (load times, CPU usage, etc.)
  - Availability (minimising the impact of maintenance/ downtime)
  - Scalability (handling more users, more data, more operations, etc.)
- **User Interface (UI):** On-screen appearance requirements (look & feel)

UI Requirements are high level

- Do not waste your time with minor details
- Focus on the important aspects of the interface
- Hand-drawing the proposed interface forces you to economize and focus on the most important features



UI artefacts/outputs

- There are 3 types of artefacts we can produce for UI requirements gathering:
  - Wireframes
  - Mockups
  - Prototypes

UI Artefacts

	Wireframe	**Mockup**	**Prototype**
What	A **quick sketch** to convey the high-level concept of new product functionality	A realistic **visual design** that resembles what the new product functionality will look like	**Interactive simulation** of new product functionality
Purpose	To gain consensus and collect internal feedback on how new functionality will work	To facilitate more detailed critiques of visual elements and functionality so changes can be made	To collect feedback by user testing the real experience
Design fidelity	Low	Middle	High
Included elements	The format and structure of content	Additional visual elements like logos, colors, and icons	Final interactive elements and navigation
Time invested	Low	Medium	High
Creator	PM or UX Designer	UX Designer	UX Designer

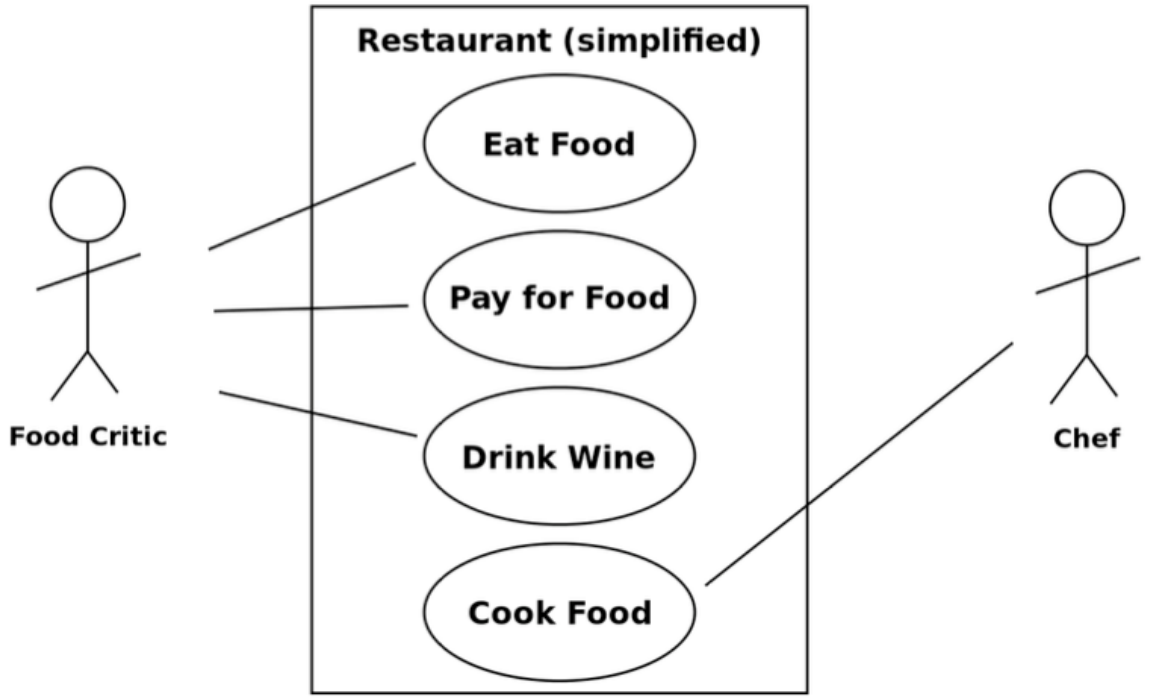
Requirements Engineering

- Requirements Gathering
  - Helps the customer to define what is required: what is to be accomplished, how the system will fit into the needs of the business, and how the system will be used on a day-to-day basis
- Requirements Analysis
  - Refining and modifying the gathered requirements
  - Not necessarily sequential – some back and forth between devs and client
- Requirements Specification
  - Documenting the system requirements in a semi-formal or formal manner to ensure clarity, consistency, and completeness
  - e.g. User stories, UML diagrams, acceptance tests, etc.
  - Ensures traceability

How do we gather & express these requirements?

Use Cases

- A use case is a list of actions or event steps typically defining the interactions between a role and a system to achieve a goal
- Represented in UML with **use case diagrams**
  - Ovals = use cases
  - Stick figures = actors
  - Boxes = system boundary
  - Lines = associations



Example Requirements (home Security)

Identifier	Priority	Requirement
REQ1	5	The system shall keep the door locked at all times, unless commanded otherwise by authorized user. When the lock is disarmed, a countdown shall be initiated at the end of which the lock shall be automatically armed (if still disarmed).
REQ2	2	The system shall lock the door when commanded by pressing a dedicated button.
REQ3	5	The system shall, given a valid key code, unlock the door and activate other devices.
REQ4	4	The system should allow mistakes while entering the key code. However, to resist “dictionary attacks,” the number of allowed failed attempts shall be small, say three, after which the system will block and the alarm bell shall be sounded.
REQ5	2	The system shall maintain a history log of all attempted accesses for later review.
REQ6	2	The system should allow adding new authorized persons at runtime or removing existing ones.
REQ7	2	The system shall allow configuring the preferences for device activation when the user provides a valid key code, as well as when a burglary attempt is detected.
REQ8	1	The system should allow searching the history log by specifying one or more of these parameters: the time frame, the actor role, the door location, or the event type (unlock, lock, power failure, etc.). This function shall be available over the Web by pointing a browser to a specified URL.
REQ9	1	The system should allow filing inquiries about “suspicious” accesses. This function shall be available over the Web.

User stories



WHO-WHAT-WHY



FOCUS ON USER BENEFITS
RATHER THAN SYSTEM BENEFITS



MUST HAVE ONE OR MORE
ACCEPTANCE TESTS ATTACHED
(TEST-DRIVEN DEVELOPMENT)



PREFERRED TOOL IN AGILE
METHODOLOGY

As a tenant, I can unlock the doors to enter my apartment.

user role (who)

capability (what)

business value (why)

How do we know that a requirement has been met?

Acceptance testing

What is an Acceptance Test?

An **acceptance test** case specifies, for a given “situation” or “context” (defined by current system inputs), the output or behavior the system will produce in response

- User Acceptance Test – Works for user (user focused)
- Operational Acceptance Test – operational readiness (system focused)

Acceptance Testing

- Acceptance tests are a means of assessing that the requirements are met as expected
- May be conducted by the client
  Describe whether the system will pass or fail a test given specific input values
- Cannot ever guarantee 100% coverage of all usage scenarios, but systematic approach can increase the degree of coverage

User Acceptance Test

- Verifies that the software solution works for the user
- Given-When-Then

User Story	As a tenant, I can unlock the doors to enter my apartment.
UAT	**Given** I am the tenant and I have the correct password, **when** I fill in my password **then** my door should unlock.
Non-Functional Tests	The system should respond in under 1 second to any password input

Requirements Analysis

- Requirements should be
  - **Documented** – recorded for future use
  - **Traceable** – who said what, which meeting, when was this designed, etc.
  - **Actionable** – break requirements into tasks that can be implemented
  - **Testable** – acceptance tests
  - **Related to business needs** – user stories (who-what-why)



Requirements Analysis

- Refinement of client requirements
  - Need to evaluate feasibility and realism
- Need to explicitly identify business policies (business rules)
  - Helps involve client and other stakeholders in designing a solution
  - Helps anticipate future changes in policy so that mechanisms can be implemented that future-proof code and allow for quick substitutions of policy
  - Helps identify and elicit further requirements

Requirements Analysis: Sprint Planning Meeting



1st Half:

Whole team (devs and product owner) select product backlog items that they believe can be completed during the sprint



2nd Half

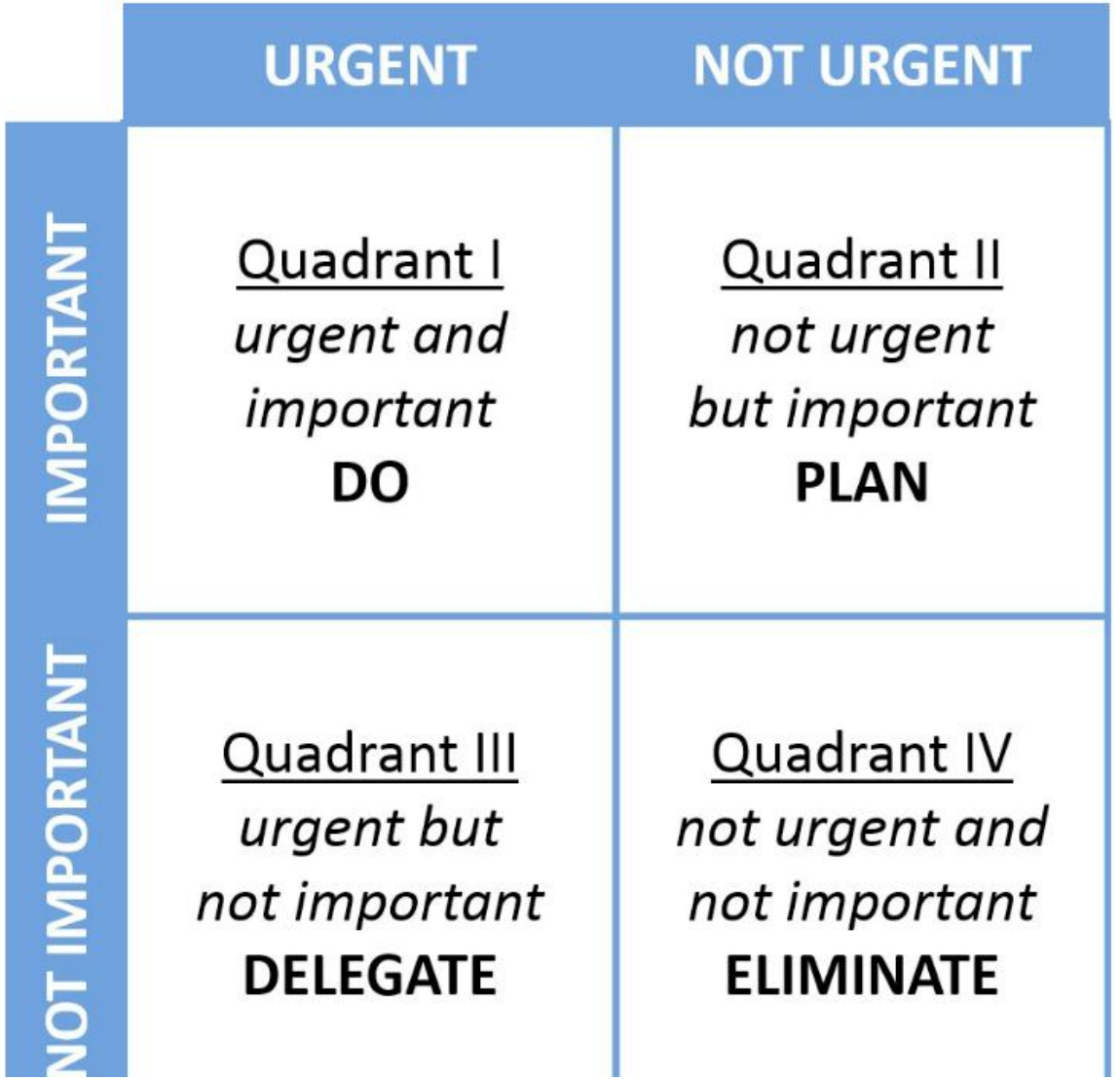
Devs break each item into tasks (actionable) required to complete those items

Tasks may include architecture, design, coding, testing, infrastructure, deployment, documentation, etc.

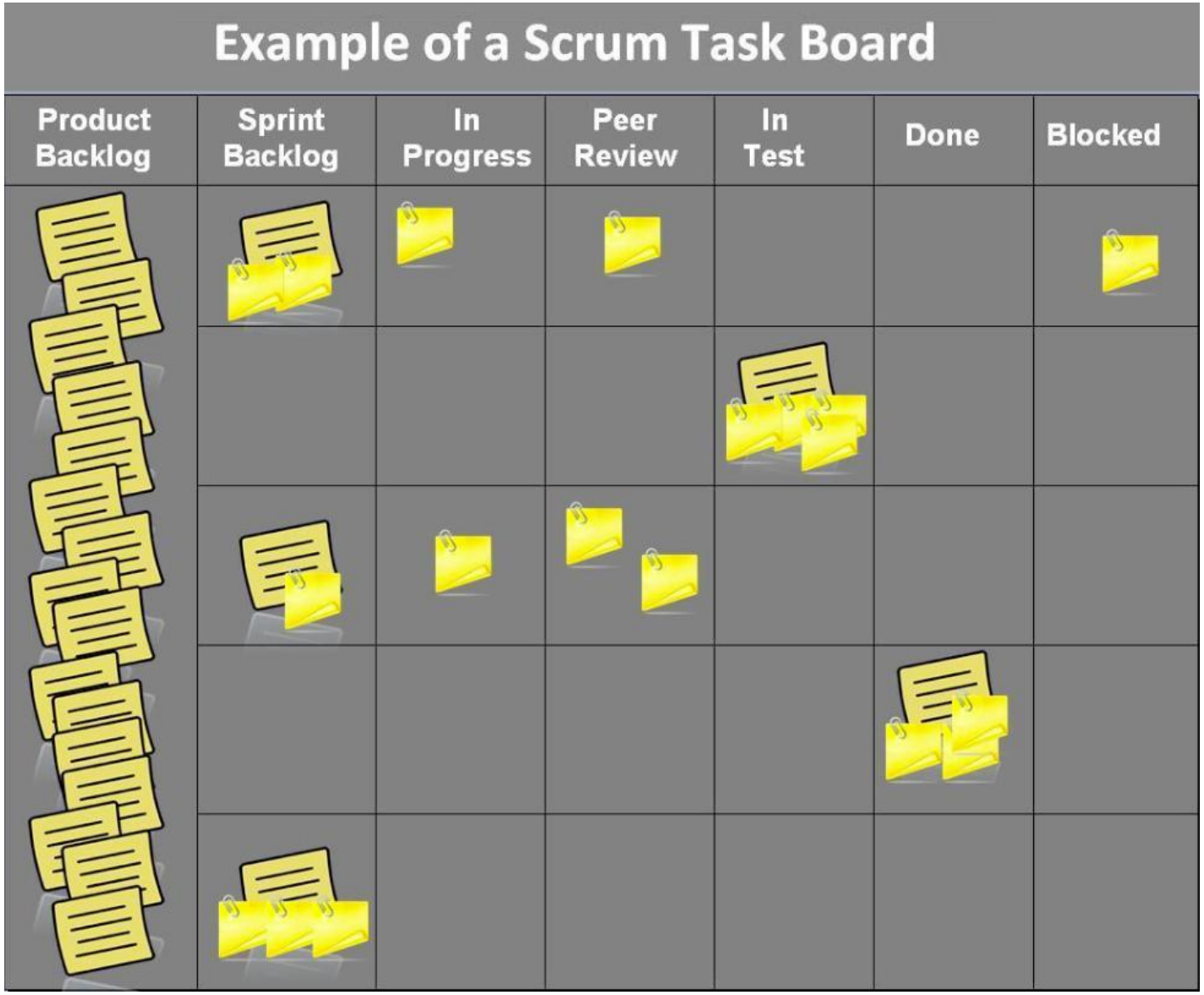
Requirements Prioritisation

Requirements prioritisation can be difficult

- There is a difference between “urgent” and “important”
- Can be difficult to assign “priority points” to requirements
- Often easier to do a rank ordering



Requirements Prioritisation



How do we know how long it will take us?



Requirements Sizing

- Often a need to work out which user stories and tasks require the most effort
- Estimated using **size points**: each user story is assigned a number of points based on its size

Identifier	User Story	Size
ST-1	As an authorized person (tenant or landlord), I can keep the doors locked at all times, so that	**4 points**
ST-2	As an authorized person (tenant or landlord), I can lock the doors on demand, so that	**3 pts**
ST-3	The lock should be automatically locked after a defined period of time, so that	**6 pts**
ST-4	As an authorized person (tenant or landlord), I can unlock the doors, so that .... (Test: Allow a small number of mistakes, say three.)	**9 points**
ST-5	As a landlord, I can at runtime manage authorized persons, so that	**10 pts**
ST-6	As an authorized person (tenant or landlord), I can view past accesses, so that	**6 pts**
ST-7	As a tenant, I can configure the preferences for activation of various devices, so that	**6 pts**
ST-8	As a tenant, I can file complaint about “suspicious” accesses, so that	**6 pts**

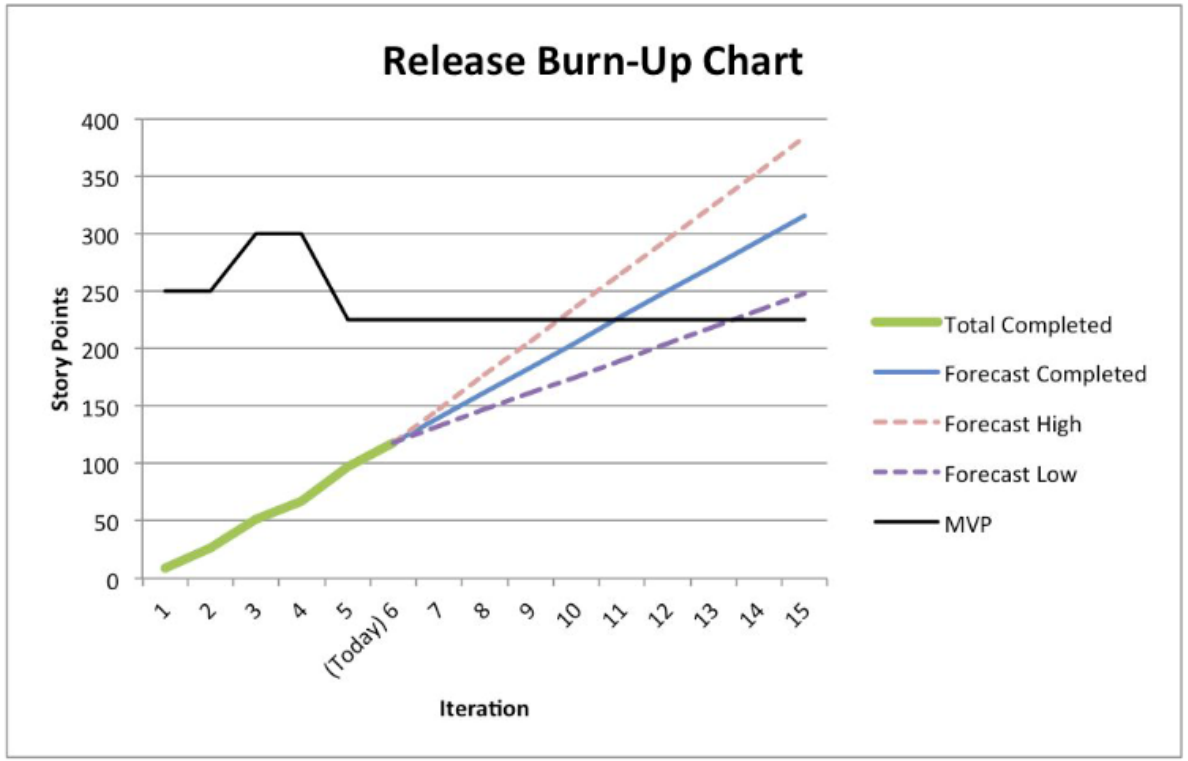
Example User Stories (Home security)

Identifier	User Story	Size
ST-1	As an authorized person (tenant or landlord), I can keep the doors locked at all times, so that	**4 points**
ST-2	As an authorized person (tenant or landlord), I can lock the doors on demand, so that	**3 pts**
ST-3	The lock should be automatically locked after a defined period of time, so that	**6 pts**
ST-4	As an authorized person (tenant or landlord), I can unlock the doors, so that .... (Test: Allow a small number of mistakes, say three.)	**9 points**
ST-5	As a landlord, I can at runtime manage authorized persons, so that	**10 pts**
ST-6	As an authorized person (tenant or landlord), I can view past accesses, so that	**6 pts**
ST-7	As a tenant, I can configure the preferences for activation of various devices, so that	**6 pts**
ST-8	As a tenant, I can file complaint about “suspicious” accesses, so that	**6 pts**

Requirements Sizing

- Total project size = sum of all user story points
- Velocity (burn rate) is estimated by looking at number of points completed per day

$$Project\ duration = \frac{Total\ Size}{Velocity}$$



Tools for Requirements Specification

- Software Tools
  - GitHub Projects
  - Monday
  - Trello
  - Taiga
  - Azure Devops
  - Notion
  - Kanboard

Done for today